Differences between MII and RMII interfaces

STackPointer64
ST Employee

on 2025-06-24 8:30 AM - edited on 2025-11-26 5:13 AM by Laurids_PETERSEN

Summary

In STM32-based Ethernet designs, connecting the internal media access controller (MAC) to an external physical layer transceiver (PHY) requires a hardware interface. The two most common interfaces used for this are:

- **MII**: Media-independent interface
- **RMII**: Reduced media-independent interface

Both serve the same purpose, facilitating data transmission between the MAC and PHY, but they differ significantly in pin count, speed, clocking, and ease of use.

This article breaks down their differences and guides you on when and why to choose one over the other in STM32-based applications.

1. Quick summary table

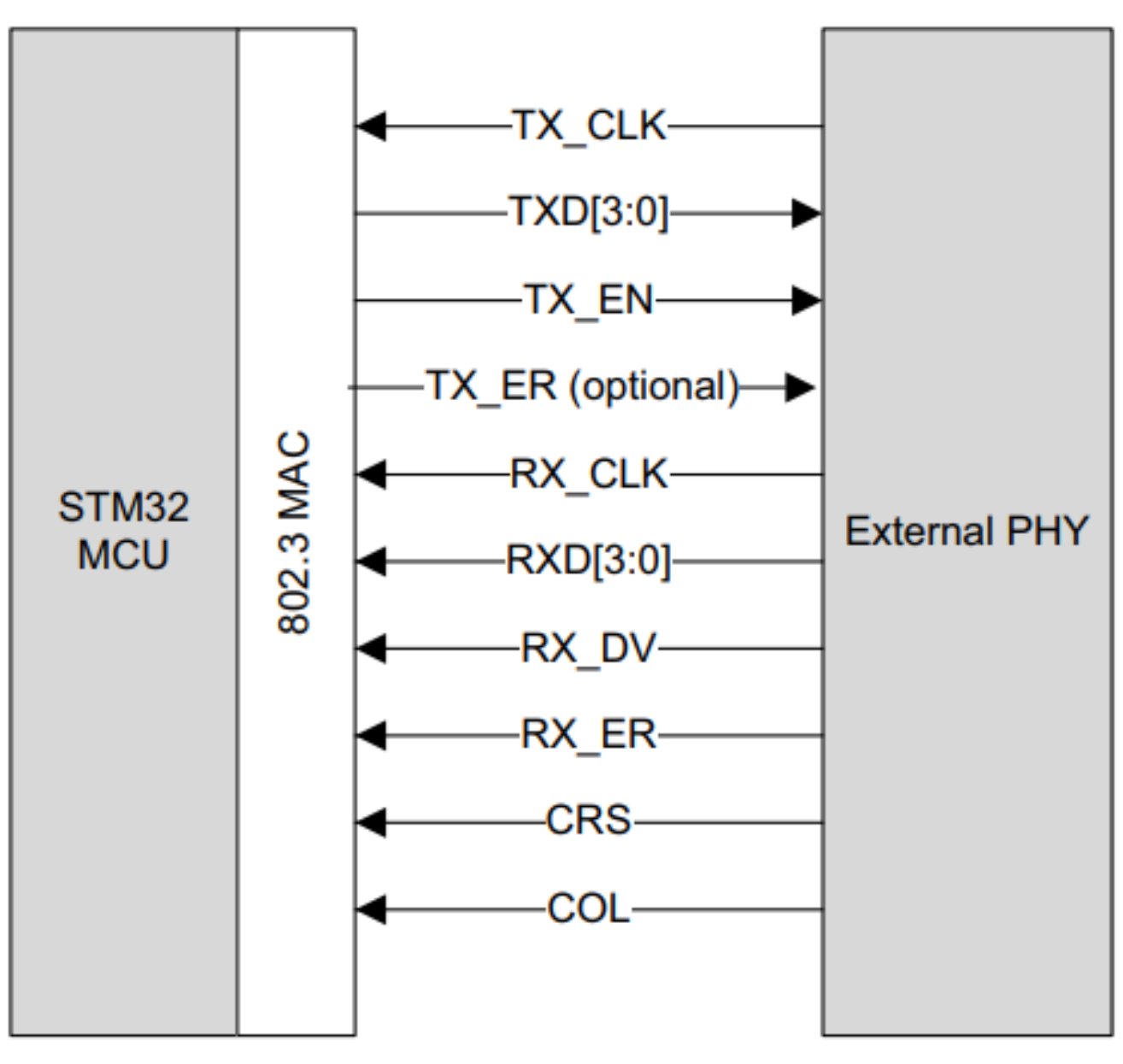
Feature	MII	RMII
Full name	Media-independent interface	Reduced media-independent interface
Supported speeds	10 Mbps & 100 Mbps	10 Mbps & 100 Mbps
Data bus width	4-bit Tx and 4-bit Rx	2-bit Tx and 2-bit Rx
Clock frequency	25 MHz (for 100 Mbps)	50 MHz
Clock direction	Separate Tx & Rx clocks	Shared 50 MHz clock
Number of required signals	~16	~8
PCB complexity	Higher	Lower
Power consumption	Higher	Lower
Used in	FPGAs, high-end MCUs, legacy PHYs	STM32 MCUs, IoT boards

2. Defining MII and RMII

MII and RMII are IEEE 802.3 standards that define how an Ethernet MAC communicates with a PHY chip over a digital interface, before signals are converted to analog Ethernet voltages for transmission on the cable.

- Media-independent interface (**MII)** defines the interconnection between the MAC sublayer and the PHY for data transfer at 10 Mbit/s and 100 Mbit/s.
- Reduced media-independent interface (**RMII)** is a streamlined version of MII that reduces the pin count by serializing data paths and saving valuable GPIOs.

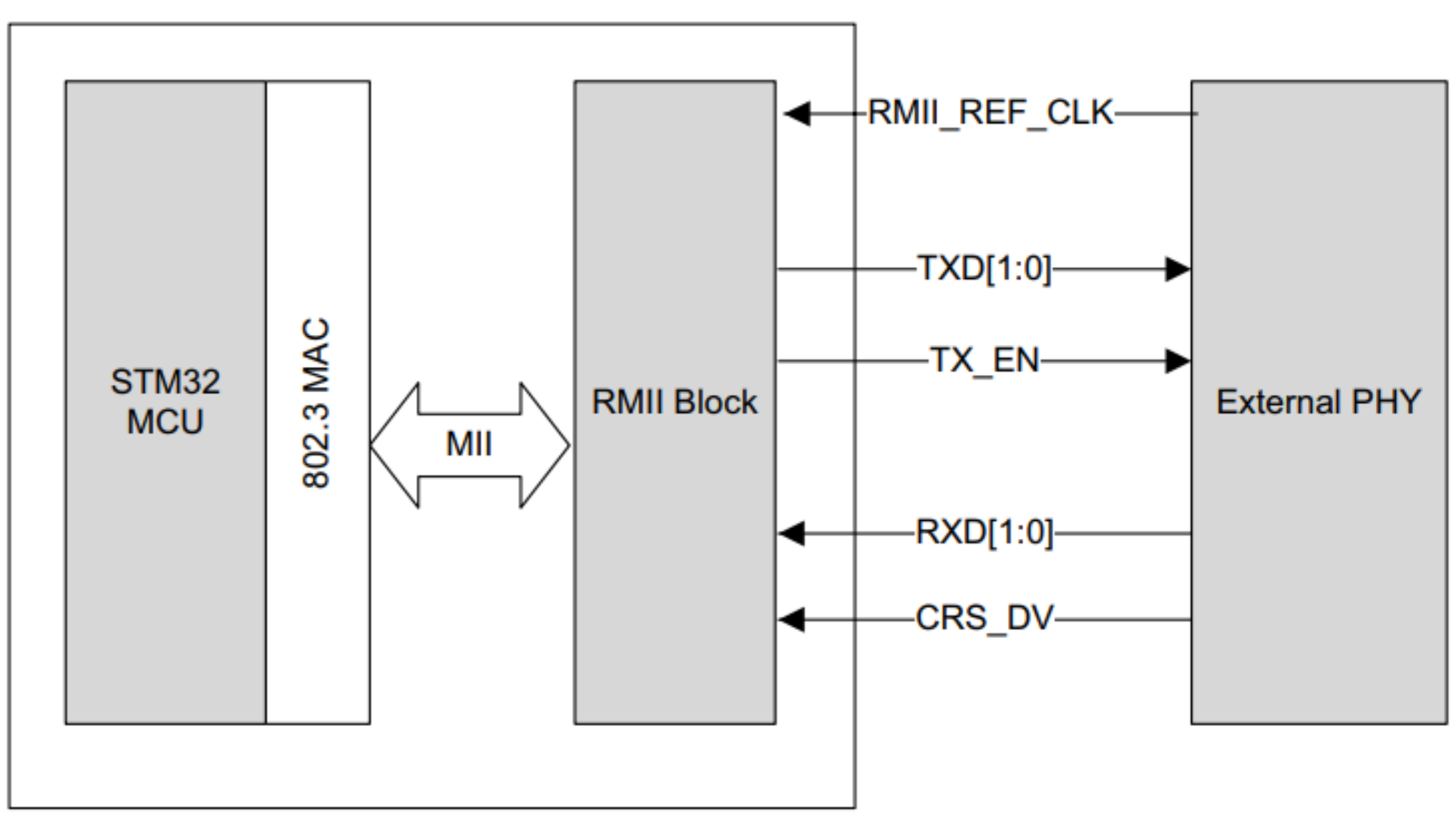
3. MII signals



RM0481 figure 822 page 2768

- **Transmit (Tx)**:
  - **TXD[3:0]** (transmit data): A bundle of four data signals driven synchronously by the MAC sublayer and qualified (valid data) on the assertion of the TX_EN signal.
  - **TX_EN** (transmission enable): Indicates that the MAC is presenting nibbles on the MII for transmission.
- **Receive (Rx)**:
  - **RXD[3:0]** (receive data): RXD is a bundle of four data signals driven synchronously by the PHY and qualified (valid data) on the assertion of the RX_DV signal.
  - **RX_DV** (receive data valid): Indicates that the PHY is presenting recovered and decoded nibbles on the MII for reception.
  - **RX_ER** (receive error): indicates to the MAC sublayer that an error was detected somewhere in the frame.
- **Clocks**:
  - **TX_CLK**: A continuous clock that provides the timing reference for Tx data transfers.
  - **RX_CLK**: A continuous clock that provides the timing reference for Rx data transfers.
- **Control**:
  - **CRS** (carrier sense): This signal is asserted by the PHY when either the transmit or receive medium is non-idle.
  - **COL** (collision detection signal): This signal must be asserted by the PHY upon detection of a collision on the medium and must remain asserted while the collision condition persists.

4. RMII signals



RM0481 figure 823 page 2770

- **Transmit (Tx)**:
  - **TXD[1:0]** (transmit data)
  - **TX_EN** (transmit data enable): When high, this bit indicates that valid data are being transmitted on TXD[1:0].
- **Receive (Rx)**:
  - **RXD[1:0]** (receive data)
  - **CRS_DV**: carrier Sense (CRS) and RX_Data Valid (RX_DV) multiplexed on alternate clock cycles.
- **Clock**:
  - **RMII_REF_CLK**: Single 50 MHz clock shared between MAC and PHY

5. Clocking differences

Interface	Clocking method	STM32 implication
MII	Separate Tx and Rx clocks	Requires more clock inputs, complex routing
RMII	Single 50 MHz reference clock	Shared by MCU and PHY; simpler layout

6. When to use MII vs RMII

Use case	Recommended interface
STM32 IoT board or dev kit	RMII
Custom board with minimal GPIOs	RMII
Application requires 10 Mbps fallback	MII
High-frequency EMI-sensitive design	RMII (with layout care)
Interfacing with older or legacy PHYs	MII

7. Troubleshooting tips

Problem	Probable cause of problem	Solution
No Ethernet link detected	Missing or unstable 50 MHz REF_CLK on STM32	Verify clock source and stability
MAC not receiving packets	Rx pins misconfigured	Check pinout and CubeMX configuration
Random PHY resets	PHY reset pin floating or undervolted	Ensure proper pull-up and power supply

Related links

- Reference manual 0481: STM32H523/33xx, STM32H562/63xx, and STM32H573xx Arm®-based 32-bit MCUs

Ethernet

3 Kudos

Version history

Last update:
2025-11-26 5:13 AM

Updated by:
Laurids_PETERSEN

Contributors

STackPointer64

About STMicroelectronics
Who we are
Investor relations
Sustainability
Innovation & technology
Careers
Blog
General terms and conditions

Connect with us
Contact ST offices
Find sales offices & distributors
Community
Newsroom
Events & trainings
AI transparency

Browse
Shortcuts
Sitemap

Compliance, ethics & privacy
Ethics and compliance
ST ethics hotline
Privacy portal

Follow us

All rights reserved © 2025 STMicroelectronics | Terms of use | Sales terms & conditions | Trademarks | Privacy portal | Manage cookies